MicroROM-NPU

RTL Module Selection and Design Rationale

Revision 0.2 | Updated for the active 48-bit microroutine architecture

1. Design assumptions

- Standalone execution engine for the selected RV32 instruction subset; not a core sourced from an existing project and not a coprocessor attached to another CPU.
- 2D vertical constant MicroROM: routine_id[3:0] and micro_step[4:0], with 48-bit microinstructions.
- One architectural PC; no dedicated IPC or TMP register. MAR retains the instruction address until safely reused.
- Instruction and data memory operations use explicit request/response completion semantics. The sequencer does not advance a micro-step while the active operation is incomplete.
- Constant ROM is initially intended for standard-cell synthesis. No FPGA-specific ROM inference is assumed.
- Neural demonstration uses signed INT8 activations/weights, signed INT32 bias/accumulator/output, and a repeated-add strategy without a RISC-V MUL instruction.

2. RTL module inventory

#	RTL module / package	Responsibility and reason
1	microrom_pkg.sv (package, not a module)	Shared field slices, UOP/selector/immediate/memory/condition/NEXT codes, routine IDs, and packed-word helper constants. Single source of truth.
2	micro_rom_2d	Combinational constant lookup by routine ID and local step; default/safe NOP for unused slots, with invalid routine/illegal slot detection specified explicitly.
3	microsequencer	Holds routine ID, local micro-step, completion gating, JUMP/COND/DISPATCH/RETURN/RESTART behavior, and one-level µRET state for CALL_MUL.
4	microinstruction_decoder	Extracts the 48-bit fields, validates field combinations and reserved bits, and gates side effects before commit.
5	instruction_decoder	Decodes the eight selected RV32 encodings, extracts fields/immediates, signals instruction_valid, and produces the routine ID.
6	instruction_register	Holds the fetched 32-bit IR word; read selectors RS1/RS2/RD derive actual register indices from IR.
7	architectural_register_file	32-bit x0-x31 bank. x0 returns zero and ignores writes.
8	internal_register_bank	Stores A, B, ACC, PROD, CTR, MDR, MAR, and SIGN. TMP is not in the current bank. PC and IR are separate modules/state.
9	alu	32-bit add/negate/compare and selected immediate operations. Supplies the combinational target result when PC_CTRL=10.
10	program_counter	The only architectural PC; implements hold, advance by 4 bytes in this version, and load ALU result. PC updates are exclusive per clock.
11	memory_interface	Coordinates instruction read and LB/LW/SW data transactions; handles request acceptance, response wait, one-time capture, access fault, and completion.
12	writeback_unit	Routes UOP results to architectural/internal destinations; resolves RS1/RS2/RD dynamic selectors and blocks writes to x0.
13	microrom_npu_top	Integration boundary for clock/reset, memory ports, start/program execution controls, and observable status. Connects modules for simulation and synthesis.

This is 12 hardware modules plus the shared package (13 numbered entries above if the package is counted as a file; the hardware module count is 12). The package is not counted as a synthesizable module.

3. Why this partition is used

Requirement	Decision	Rationale
Keep routines logically isolated	2D MicroROM plus routine-local step	A local branch stays within its routine; routine changes use DISPATCH, RETURN, or RESTART.
Separate architectural and microprogram flow	Program counter plus microsequencer	PC tracks architectural instructions; routine ID and micro-step track microcode.
Support dynamic operands	IR plus RS1/RS2/RD selector codes	The same fixed routine handles all legal register numbers and immediate values.
Handle memory latency	Memory interface + step_complete	The sequencer holds during request/response stalls and avoids duplicate requests.
Prevent accidental side effects	Microinstruction decoder / legality gate	Invalid encodings must suppress register writes, memory requests, flag updates, and PC changes.
Simplify branch target path	MAR holds original instruction address; ALU feeds PC	Avoids both IPC and TMP registers; branch/jump routine calculates target before MAR reuse.
Target ASIC standard-cell synthesis	Constant case/table ROM initially	Allows synthesis tools to optimize fixed control constants without relying on FPGA block RAM.

4. Interface contracts to freeze before RTL

MicroROM and sequencer

• micro_rom_2d returns a 48-bit word for routine_id/local_step. All 320 positions in IDs 0-9 must have documented defined contents (operational word or explicit safe default); IDs 10-15 are invalid.

• The sequencer evaluates NEXT only after step_complete. NEXT_ADDR is a 5-bit local step; NEXT=DISPATCH enters a decoded routine at step zero; NEXT=RESTART enters Routine 0 at step zero.

• CALL_MUL is legal only when NEXT=SEQ and no saved call is outstanding. µRET stores caller routine ID and next local step. Nested calls are unsupported.

Memory and datapath

• Memory requests remain valid until accepted. After acceptance, the controller remembers that fact and waits for response; it does not reissue the same request. Returned data is captured exactly once.

• MEM_CTRL=100 denotes instruction read; MEM_CTRL=001/010/011 denotes signed-byte read, word read, and word write respectively. Error/fault response suppresses normal completion and must reach a defined error state.

• UOP=NOP means no ordinary datapath operation/write; it does not prohibit independently selected memory, PC, or sequencer actions.

• PC_CTRL=00 holds PC, 01 advances by 4 bytes in the current subset, 10 loads ALU result, and 11 is illegal. At most one PC action can commit per clock.

• IR fields drive dynamic selectors RS1 (41), RS2 (42), RD (43); selectors are not additional registers. PC selector 45 is read-only. Selector 34/TMP and selector 46 are reserved.

5. Recommended implementation order

Step	Deliverable	Acceptance gate
1	Freeze microrom_pkg.sv and field/code tables.	Field widths total 48; pack/unpack tests round-trip; reserved codes and canonical unused fields documented.
2	Validate instruction_decoder.	Directed tests for all eight instructions, immediate sign extension, bad funct/opcode encodings, valid flag and routine IDs.
3	Finish Routine 0, LB and LW words; formulate SW, ADD, ADDI, BNE, BGE, JAL, routine 9.	All steps and transitions listed; target steps in bounds; field combinations legal.
4	Implement micro_rom_2d.	All entries defined; representative outputs match the approved word table.
5	Implement microinstruction decoder and microsequencer.	Test stalls, dispatch, local branches, restart, call/return, illegal word shutdown.
6	Implement register banks, ALU, PC, memory interface and writeback.	Test dynamic selectors, x0, load sign extension, signed comparisons, PC exclusivity and memory backpressure.
7	Integrate top level and run end-to-end programs.	Architectural reference checks pass; only then record synthesis area/timing results.

6. Verification plan

- Microcode encoding: round-trip pack/decode of all 48 bits, every enumerated UOP/NEXT/MEM_CTRL/COND/FLAGS_CTRL value, and illegal combinations.
- Memory: request not-ready for multiple cycles, response delayed, response same cycle as acceptance if supported, error response, byte sign extension, word alignment, no duplicate reads/writes.
- Control flow: sequential advance only after completion; invalid instruction never dispatches; dispatch/RESTART/RETURN and local jump targets; CALL_MUL with/without outstanding µRET.
- Architectural state: x0 constant zero; RS1/RS2/RD dynamic fields; PC holds/advances/loads target exclusively; branch target is based on MAR; JAL link value is advanced PC.
- ISA subset: ADD/ADDI wraparound; LB values 0x7F/0x80/0xFF; LW/SW aligned operations; BNE/BGE taken and not taken; JAL target and link.
- Neural workload: signed INT8 values, repeated-add product, bias, ReLU, loop boundaries, software reference comparison.

7. Open item that must remain visible

Routine 9 is planned as a repeated-add multiplication helper and CALL_MUL provides a one-level entry mechanism, but the selected architectural instruction subset has no MUL opcode and none of the already-defined routines calls routine 9. The application-level or microcode-level caller interface must be chosen explicitly before routine 9 and the top-level program are treated as frozen.

Document status: proposed module partition, not completed RTL. Module ports, reset sequencing, memory faults, and all routine words still require implementation and verification.

Module-freeze acceptance gate

Area	Required evidence
Control store	All 320 entries for routine IDs 0-9 are defined; unused slots have explicit safe contents; invalid routine IDs are detected.
Encoding	Independent encoder/decoder test confirms every packed word matches its symbolic field table.
Sequencer	Stalls, local branches, DISPATCH, RESTART, RETURN, CALL_MUL and error-stop behavior pass directed tests.
Datapath and memory	x0, dynamic selectors, sign extension, PC exclusivity, memory backpressure, data faults and write suppression pass directed tests.
Integration	End-to-end instruction programs match a software reference before synthesis claims are reported.